

SIH 25162 - IC Verification Algorithm

Automated Optical Inspection for Counterfeit IC Detection

ALGORITHM OVERVIEW

Our system uses a hybrid two-algorithm approach to verify IC authenticity:

1. **Algorithm A:** RegEx-Based Verification (Direct Pattern Matching)
2. **Algorithm B:** Aho-Corasick-Based Verification (Intelligent Auto-Detection)

Both algorithms use a 3-stage pipeline:

- **Stage 1:** IC Detection (YOLOv8)
- **Stage 2:** Text Extraction (Multi-Pass Tesseract OCR)
- **Stage 3:** Pattern Verification (RegEx/Aho-Corasick)

STAGE 1: IC DETECTION (YOLO)

Input:

- Raw image from camera/upload (RGB, any resolution)

Process:

1. Load YOLOv8 trained model (custom-trained on IC datasets)
2. Run object detection on image
3. Extract bounding boxes of detected IC components
4. Apply padding (10-15px) around detected regions
5. Crop IC marking areas for OCR

Output:

- Cropped IC marking images (numpy arrays)
- Bounding box coordinates
- Detection confidence scores

Technical Details:

- **Model:** YOLOv8n (custom fine-tuned)
- **Confidence Threshold:** 0.25
- **IOU Threshold:** 0.45
- **Device:** CPU/CUDA (auto-detected)

Stage 2 : Text Extraction and Multi-pass OCR

Input:

- Cropped IC marking image (from Stage 1)

Process:

Step 2.1: Image Preprocessing

1. Convert BGR → Grayscale
2. Apply bilateral filter (noise reduction while preserving edges)
3. Normalize intensity (0-255 range)
4. Apply adaptive thresholding (binary image)

Step 2.2: Multi-Pass OCR Execution

Pass 1: Full Alphanumeric

Purpose: Extract part codes and lot codes

Whitelist: A-Z, 0-9

PSM Mode: 6 (uniform text block)

Example Output: "STM32F030C8T6 2347 A3B5C"

Pass 2: Numeric Only

Purpose: Extract date codes (prevents letter misreads like "O" → "0")

Whitelist: 0-9

PSM Mode: 6

Example Output: "2347"

Pass 3: Alpha Only

Purpose: Extract logo hints and OEM names

Whitelist: A-Z

PSM Mode: 6

Example Output: "STMI"

Output: JSON

```
{  
  "full_alphanumeric": {"text": "STM32F030C8T6 2347 A3B5C", "confidence": 0.87},  
  "numeric_only": {"text": "2347", "confidence": 0.92},  
  "alpha_only": {"text": "STMI", "confidence": 0.81}  
}
```

Technical Details:

- OCR Engine: Tesseract 4. x (OEM 3 - Default LSTM)
- Preprocessing: Bilateral Filter + Adaptive Thresholding
- Confidence Calculation: Mean of valid word confidences (normalized 0-1)

STAGE 3: PATTERN VERIFICATION

ALGORITHM A: RegEx-Based Verification (Direct Mode)

Use Case: When operator knows which IC is being scanned

Input: JSON

```
{  
  "ocr_results": {  
    "full_alphanumeric": {"text": "STM32F030C8T6 2347 A3B5C", "confidence": 0.87},  
    "numeric_only": {"text": "2347", "confidence": 0.92}  
  },  
  "user_selected_part_id": "stm32f030c8t6"  
}
```

Process:

Step 3A. 1: Load KB Entry

1. Load Knowledge Base entry for "stm32f030c8t6"
2. Extract OEM-defined regex patterns:
 - part_code: "\\bSTM32F030C8T6\\b"
 - date_code: "\\b[0-9]{2}[0-5][0-9]\\b"
 - lot_code: "\\b[0-9A-Z]{3,6}\\b"
3. Compile patterns into regex objects

Step 3A.2: Field Extraction

Apply regex patterns on OCR text:

Pattern 1: part_code regex on full_alphanumeric. text

→ Extract: "STM32F030C8T6"

→ Match:  TRUE

Pattern 2: date_code regex on numeric_only.text

→ Extract: "2347"

→ Match:  TRUE

→ Validate: Year=2023, Week=47 (valid, not future)

Pattern 3: lot_code regex on full_alphanumeric.text

→ Extract: "A3B5C"

→ Match:  TRUE

Logo Hint Check (optional):

→ Search "STMI" in alpha_only.text

→ Match:  TRUE (bonus +0.05)

Step 3A.3: Scoring & Verdict Calculation

Match Count: 3/3 

OCR Confidence: 0.87

Logo Bonus: +0.05

Date Validation: PASS (not future date)

Base Score Calculation:

- 3 matches → 0.90 base confidence
- OCR conf (0.87) → no penalty
- Logo found → +0.05
- Final Score: 0.95

Verdict Threshold Logic:

- 3/3 matches + score ≥ 0.85 → GENUINE
- 2/3 matches + score ≥ 0.50 → UNCERTAIN
- 0-1/3 matches → FAKE

Final Verdict: GENUINE (0.95 confidence)

Output:

```
{
  "verdict": "GENUINE",
  "confidence_score": 0.95,
  "algorithm_used": "regex",
  "matches": {
    "part_code_match": true,
    "date_code_match": true,
    "lot_code_match": true,
    "logo_hint_match": true
  },
  "extracted_fields": {
    "part_code": "STM32F030C8T6",
    "date_code": "2347",
    "lot_code": "A3B5C",
    "date_validation": {
      "valid": true,
      "year": 2023,
      "week": 47,
      "flags": []
    }
  },
  "oem_info": {
    "oem": "STMicroelectronics",
    "part_number": "STM32F030C8T6",
    "package": "LQFP-48 / QFN-48"
  }
}
```

ALGORITHM B: Aho-Corasick-Based Verification (Auto-Detection Mode)

Use Case: Fully automated - system identifies which IC is present

Input:

```
{  
  "ocr_results": {  
    "full_alphanumeric": {"text": "STM32F030C8T6 2347 A3B5C", "confidence": 0.87},  
    "numeric_only": {"text": "2347", "confidence": 0.92},  
    "alpha_only": {"text": "STMI", "confidence": 0.81}  
  },  
  "user_selected_part_id": null  
}
```

Process:

Step 3B.1: Token Indexing (Pre-computed at startup)

Build Aho-Corasick automaton from KB:

For each KB entry:

1. Extract tokens from part_number:

- "STM32F030C8T6" → ["STM32F030C8T6"]
- "74HC14 / 74HCT14" → ["74HC14", "74HCT14"]

2. Extract alternatives from part_code pattern:

- "\\bSTM32F030C8T6\\b" → ["STM32F030C8T6"]
- "\\b(74HC14|74HCT14)\\b" → ["74HC14", "74HCT14"]

3. Include logo_hint:

- "STMI", "TEXA", "WINB", etc.

4. Add to automaton:

- Token "STM32F030C8T6" → maps to part_id "stm32f030c8t6"
- Token "STMI" → maps to part_id "stm32f030c8t6"

Total tokens indexed: ~50-100 (for 16 KB entries)

Step 3B.2: Candidate Detection

Run Aho-Corasick string matching on OCR text:

Input Text: "STM32F030C8T6 2347 A3B5C STMI"

Automaton finds:

- Token "STM32F030C8T6" at position 0
→ Candidate: "stm32f030c8t6"
- Token "STMI" at position 20
→ Candidate: "stm32f030c8t6"

Candidate Set: {"stm32f030c8t6"}  Single candidate!

Step 3B.3: Candidate Scoring

For each candidate, apply RegEx verification (same as Algorithm A):

Candidate: "stm32f030c8t6"

- part_code match: 
- date_code match: 
- lot_code match: 
- Score: 3/3 → Confidence 0.95

(If multiple candidates exist, rank by score and return top match)

Step 3B.4: Ambiguity Handling

If multiple candidates have similar scores:

- Candidate A: "stm32f030c8t6" → Score 3/3 (0.95)
- Candidate B: "stm32f030c6t6" → Score 2/3 (0.65)

Action:

- If score difference ≥ 0.20 → Return best candidate
- If score difference < 0.20 → Verdict = "MULTIPLE_CANDIDATES"
→ Flag for admin review

Output (Single Candidate):

```
{  
  "verdict": "GENUINE",  
  "confidence_score": 0.95,  
  "algorithm_used": "aho_corasick",  
  "detected_part_id": "stm32f030c8t6",  
  "candidate_parts": ["stm32f030c8t6"],  
  "matches": {  
    "part_code_match": true,  
    "date_code_match": true,  
    "lot_code_match": true  
  },  
  "extracted_fields": { /* same as Algorithm A */ },  
  "oem_info": { /* same as Algorithm A */ }  
}
```

Output (Multiple Candidates - Ambiguous):

```
{
  "verdict": "MULTIPLE_CANDIDATES",
  "confidence_score": 0.0,
  "requires_admin_review": true,
  "algorithm_used": "aho_corasick",
  "all_candidates": [
    {
      "part_id": "stm32f030c8t6",
      "oem": "STMicroelectronics",
      "match_score": 3,
      "confidence": 0.87
    },
    {
      "part_id": "stm32f030c6t6",
      "oem": "STMicroelectronics",
      "match_score": 2,
      "confidence": 0.65
    }
  ]
}
```

VERDICT DETERMINATION LOGIC

Scoring Matrix:

Match Count	OCR Confidence	Date Valid	Logo Match	Base Score	Final Verdict
3/3	≥ 0.80	✓ Yes	✓ Yes	0.90 + 0.05	GENUINE (0.95)
3/3	≥ 0.80	✓ Yes	✗ No	0.90	GENUINE (0.90)
3/3	< 0.60	✓ Yes	✓ Yes	0.75 + 0.05	UNCERTAIN (0.80)
2/3	≥ 0.70	✓ Yes	✓ Yes	0.60 + 0.05	UNCERTAIN (0.65)
2/3	≥ 0.70	✗ Future Date	-	0.30	FAKE (0.30)
1/3	Any	✓ Yes	-	0.35	FAKE (0.35)
0/3	Any	Any	-	0.15	FAKE (0.15)
Any	Any	✗ Future Date	-	0.20	FAKE (0.20)

Special Cases:

1. Future Date Detection:

- If date_code year/week is in the future → Auto-FAKE
- Confidence capped at 0.30 maximum

2. Multiple Candidates (Aho algorithm only):

- If 2+ candidates with score difference < 0.20 → MULTIPLE_CANDIDATES
- Requires admin review

3. Low OCR Confidence:

- If average OCR confidence < 0.50 → Reduce final score by 0.15
- Flags as "poor image quality"

TECHNICAL SPECIFICATIONS

Performance Metrics:

- Detection Time: ~150-300ms per image (YOLO)
- OCR Time: ~200-400ms (3 passes)
- Verification Time: ~5-10ms (RegEx) / ~10-20ms (Aho-Corasick)
- Total Pipeline: <1 second per IC

Accuracy Benchmarks (Target for SIH):

- True Positive Rate: $\geq 95\%$ (genuine ICs correctly identified)
- True Negative Rate: $\geq 90\%$ (fake ICs correctly detected)
- False Positive Rate: $\leq 5\%$ (fake ICs marked genuine)
- False Negative Rate: $\leq 10\%$ (genuine ICs marked fake)

Scalability:

- Concurrent Processing: Multi-threading via `run_in_executor`
- Batch Processing: Supports 10-100 images per batch
- Database Logging: Async MongoDB writes (non-blocking)

ALGORITHM ADVANTAGES (For Judges)

1. Dual-Algorithm Robustness

- Combines deterministic (RegEx) and probabilistic (Aho-Corasick) approaches
- Operator can choose based on use case (speed vs. automation)

2. Multi-Pass OCR Accuracy

- Field-specific character whitelists reduce OCR errors
- Date codes extracted with 0-9 only (prevents "O"→"0" mistakes)

3. Future-Proof Date Validation

- Automatic detection of impossible dates (clear fake indicator)
- Validates manufacturing week (1-53 range)

4. Intelligent Auto-Detection

- Aho-Corasick enables substring matching in $O(n)$ time
- No manual part selection needed in automated mode

5. Audit Trail & Traceability

- All verifications logged to MongoDB
- Admin review queue for ambiguous cases